

Name:P Neeharika

Roll No: 2520090233

PRACTICAL 2

Task 1: File Copy Using System Calls

Develop a C program that uses the system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to another. Explain how control transitions between user space and kernel space during execution.

```
root@HP:~/shellforge# cd ~/shellforge
root@HP:~/shellforge# nano source.txt
root@HP:~/shellforge# cat source.txt
Operating System Practical Week 2
This file is being copied using Linux system calls.
root@HP:~/shellforge# nano practical2_task1.c
root@HP:~/shellforge# gcc practical2_task1.c -o practical2_task1
root@HP:~/shellforge# ./practical2_task1
File copied successfully.
root@HP:~/shellforge# cat source.txt
Operating System Practical Week 2
This file is being copied using Linux system calls.
root@HP:~/shellforge# cat destination.txt
Operating System Practical Week 2
This file is being copied using Linux system calls.
root@HP:~/shellforge# diff source.txt destination.txt
root@HP:~/shellforge# |
```

Task 2: System Call Tracing Using `strace`

Use the `strace` utility to trace all system calls generated by the command `cat sample.txt`. Analyze the sequence of system calls and identify the kernel services involved.

```
root@HP:~/shellforge# strace --version
strace -- version 6.8
Copyright (c) 1991-2024 The strace developers <https://strace.io>.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

Optional features enabled: stack-trace=libunwind stack-demangle m32-mpers mx
32-mpers
root@HP:~/shellforge# cat sample.txt
This is a sample file for Practical 2.
This file is used for system call tracing.
root@HP:~/shellforge# |
```

```

root@HP:~/shellforge# strace cat sample.txt
execve("/usr/bin/cat", ["cat", "sample.txt"], 0x7ffce7a7d698 /* 28 vars */)
= 0
brk(NULL) = 0x58b3a0197000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x73d254359000
access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=19695, ...}) = 0
mmap(NULL, 19695, PROT_READ, MAP_PRIVATE, 3, 0) = 0x73d254354000
close(3) = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\220\243\2\0\0\0\0"
..., 832) = 832
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0\0@\0\0\0\0\0\0\0\0\0\0\0"
., 784, 64) = 784
fstat(3, {st_mode=S_IFREG|0755, st_size=2125328, ...}) = 0
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0\0@\0\0\0\0\0\0\0\0\0\0\0"
., 784, 64) = 784
mmap(NULL, 2170256, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x73d254000000
mmap(0x73d254028000, 1605632, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x73d254028000
mmap(0x73d2541b0000, 323584, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1b0000) = 0x73d2541b0000
mmap(0x73d2541ff000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1fe000) = 0x73d2541ff000
mmap(0x73d254205000, 52624, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x73d254205000
mmap(NULL, 360460, PROT_READ, MAP_PRIVATE, 3, 0) = 0x761cc4727000
close(3) = 0
fstat(1, {st_mode=S_IFCHR|0600, st_rdev=makedev(0x88, 0), ...}) = 0
openat(AT_FDCWD, "sample.txt", O_RDONLY) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=82, ...}) = 0
fadvise64(3, 0, 0, POSIX_FADV_SEQUENTIAL) = 0
mmap(NULL, 139264, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x761cc4705000
read(3, "This is a sample file for Practi"... , 131072) = 82
write(1, "This is a sample file for Practi"... , 82) = 82
read(3, "", 131072) = 0
munmap(0x761cc4705000, 139264) = 0
close(3) = 0
close(1) = 0
close(2) = 0
exit_group(0) = ?
+++ exited with 0 +++
root@HP:~/shellforge#

```

Overall Observation

The practical successfully demonstrated how Linux programs interact with the operating system through system calls. File operations were performed using `open()`, `read()`, `write()`, and `close()`, while `strace` was used to observe the system calls generated by the `cat` command. The experiments demonstrated the interaction between user-space programs and Linux kernel services.